

INTERNSHIP PROJECT REPORT

ON

AI-Powered FAQ Chatbot for MPOnline Services

Internship Project - **MPOnline Limited**



GOVERNMENT
SERVICES



EASY ACCESS TO
INFORMATION



ONLINE
SERVICES



RELIABLE
SUPPORT



SUBMITTED BY

AYUSH PATEL

B.Tech (Computer Science & Engineering)
8th Semester



GUIDES

Dhruv Pandey
Trainee (SME)

Tulsiram Sir
Trainee (SME)



COLLEGE

AKS University, Satna
Department of
Computer Science & Engineering



INTERNSHIP ORGANIZATION

MPOnline Limited
Madhya Pradesh



ACADEMIC YEAR
2025 – 2026

CERTIFICATE

This certifies that the project report entitled “**AI-Powered FAQ Chatbot for MPOnline Services**” submitted by partial fulfilment of the requirement for the degree of Bachelor of Technology in Computer Science and Engineering in **Jan-June 2025-2026** AKS University, Satna is a Bonafide project work carried out by **Ayush Patel, B2255R10106223**, under my supervision. The subject of the project report has been approved by the supervisor. This is also to certify that it is his original work and no part of this project is report has been submitted for any other degree/diploma.

All the assistance help received during the course of the investigation has been duly acknowledged.

1. I am satisfied that the report presented by **Ayush Patel** is worthy of consideration for the award of the degree.
2. I certify:
 - i) That he pursued the prescribed course for the project.
 - ii) That he bears good moral character.

Place: AKSU, Satna

Date: 29/04/2026

.....
Supervisor
(Mr. Dhruv Pandey)
(Trainee /SME)

.....
Head of Department
Prof. (Dr.) Akhilesh A. Wao
(Associate Dean, Head and
Professor, CS/IT, AKS Univ.)

SELF DECLARATION

I hereby declare that the work presented in this project entitled “**AI-Powered FAQ Chatbot for MPOne Services**”, carried out during my internship at MPOne Limited, is an authentic record of my own work. This project has been completed under the guidance and supervision of the concerned mentors.

This project report is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in the Department of Computer Science & Engineering, AKS University, Satna (M.P.).

I further declare that I have not submitted this work, either fully or partially, for the award of any other degree or diploma to any other institute or university.

.....

Signature of Candidate

Ayush Patel

ACKNOWLEDGEMENT

It gives me great pleasure to express my sincere gratitude to Prof. (Dr.) **Akhilesh A. Waoo**, Head of the Department of Computer Science & Engineering, AKS University, Satna (M.P.), for providing me the opportunity to undertake this project.

I consider myself fortunate to have worked on this project titled “AI-Powered FAQ Chatbot for MPOOnline Services”, which has added significant value to my knowledge and skills.

I would also like to extend my sincere thanks to my industry mentors, **Mr. Tulsiram Sir** (SME, MPOOnline Ltd.) and **Mr. Dhruv Pandey** (Trainee SME, MPOOnline Ltd.), for their guidance, support, and practical insights during my internship at MPOOnline Limited. Their experience and mentorship played a crucial role in shaping this project.

I am deeply grateful to MPOOnline Limited for providing me the opportunity to work in a professional environment and gain real-world experience.

Finally, I would like to express my sincere thanks to my parents for their constant support, encouragement, and motivation throughout my academic journey

.

ABSTRACT

The project titled “**AI-Powered FAQ Chatbot for MPOnline Services**” is designed to provide an intelligent and efficient solution for answering user queries related to government services offered through the MPOnline platform. In the current digital era, users often face difficulties in understanding complex procedures, required documents, and application steps for various services such as income certificates, caste certificates, bill payments, and online registrations. This project aims to simplify access to such information by using an AI-driven chatbot system.

The system is developed using a **Retrieval-Augmented Generation (RAG)** approach, which ensures that responses are generated based on actual data rather than generic AI knowledge. The chatbot processes documents such as PDFs and images using **Optical Character Recognition (OCR)** to extract text. This extracted data is then divided into smaller segments (chunks) and converted into vector embeddings, which are stored in a vector database. When a user asks a question, the system retrieves the most relevant information from the stored data and generates a context-aware and accurate response using a Large Language Model (LLM).

The application is built using **FastAPI** for the backend and **React.js** for the frontend, ensuring a fast and responsive user experience. A **PostgreSQL database** is used to manage structured data such as user accounts, chat history, grievances, and feedback. The system also includes additional features such as **expert escalation**, where unresolved queries are forwarded to human experts, and a **grievance management system**, allowing users to submit and track complaints. An analytics dashboard is also provided for monitoring system performance and user interactions.

This project significantly reduces the dependency on manual support systems and improves the accessibility and efficiency of digital government services. By combining modern AI techniques with practical system design, the chatbot provides a scalable, accurate, and user-friendly solution for handling large volumes of user queries.

TABLE OF CONTENT

• Introduction	07 - 08
• Objective of The Project	09 - 10
• Scope of The Projects	11 - 12
• Definition of Problem	13 - 13
• Benefit of The Projects	14 - 14
• Main Module	15 - 16
• Technical Overview	17 - 18
• System Configuration	19 - 20
• Block Diagram	21 - 21
• DFD Diagram	22 - 22
• ER Diagram	23 - 23
• Output Screen	24 - 28
• Project Explanation	29 - 32
• Codding (Sample with explanation)	33 - 49
• Software Testing	50 - 56
• Conclusion	57 - 58
• Bibliography or Reference	59 - 59

INTRODUCTION

In today's rapidly evolving digital era, the delivery of government services has significantly shifted towards online platforms to ensure better accessibility, transparency, and efficiency. One such platform is MPOnline, which provides a wide range of services including certificate generation, bill payments, examination forms, scholarship applications, and various government-related registrations. While these services are designed to simplify processes for citizens, many users still face challenges in understanding procedures, required documents, eligibility criteria, and application steps due to the complexity of information and limited support availability.

Most users, especially those from rural or non-technical backgrounds, often struggle to navigate through different sections of the platform or interpret official instructions. They frequently depend on external help such as cyber cafes or manual support, which increases both time and cost. Additionally, traditional customer support systems are not always available, and even when they are, they may be slow or unable to handle a large volume of queries efficiently. This creates a need for an intelligent, automated, and scalable solution that can assist users in real-time.

To address these challenges, this project proposes the development of an **AI-Powered FAQ Chatbot for MPOnline Services**. The chatbot acts as a virtual assistant that can understand user queries and provide instant, accurate, and context-aware responses. Unlike traditional chatbots that rely on predefined answers, this system is built using a modern **Retrieval-Augmented Generation (RAG)** approach. This ensures that the responses are generated based on actual documents and data rather than generic or pre-trained knowledge, thereby increasing reliability and reducing the chances of incorrect information.

The system works by first processing official documents such as PDFs and images related to MPOnline services. These documents are analyzed using **Optical Character Recognition (OCR)** techniques to extract text content. The extracted text is then divided into smaller segments, known as chunks, and converted into vector representations using embedding models. These vectors are stored in a vector database, enabling efficient similarity-based search.

When a user submits a query, the system converts the query into a vector and retrieves the

most relevant chunks of information from the database. These relevant pieces of information are then passed to a Large Language Model (LLM), which generates a well-structured and contextually accurate response. This approach ensures that the chatbot provides answers that are grounded in actual data, minimizing errors and hallucinations.

In addition to answering queries, the system includes several important features to enhance user experience and system effectiveness. A **grievance management system** is integrated to allow users to submit complaints or issues if their queries are not resolved. Each grievance is assigned a unique ID, enabling users to track its status. Furthermore, an **expert escalation mechanism** is implemented, where unanswered or low-confidence queries are forwarded to human experts for manual resolution. This ensures that no query remains unresolved and maintains the reliability of the system.

The application is developed using modern technologies such as **FastAPI** for backend development, **React.js** for frontend user interface, and **PostgreSQL** for managing structured data. A vector database (FAISS) is used for efficient storage and retrieval of embeddings, and AI models such as OpenAI or Gemini are used for generating responses. This combination of technologies makes the system robust, scalable, and efficient.

Overall, this project aims to improve the accessibility and usability of MPOnline services by providing an intelligent support system that is available 24/7. It reduces the dependency on manual assistance, saves time for users, and ensures that accurate information is delivered quickly. By integrating artificial intelligence with real-world applications, the system demonstrates how modern technology can be used to solve practical problems and enhance digital service delivery.

OBJECTIVES OF THE PROJECT

The main objective of the project “**AI-Powered FAQ Chatbot for MPOnline Services**” is to develop an intelligent and automated system that can assist users in accessing accurate information related to various government services. The specific objectives of this project are as follows:

1. To Develop an AI-Based Chatbot

The primary objective is to design and implement an AI-powered chatbot that can understand user queries and provide accurate, relevant, and context-aware responses related to MPOnline services.

2. To Implement Retrieval-Augmented Generation (RAG)

To use a RAG-based approach that retrieves relevant information from documents and generates responses based on actual data, ensuring high accuracy and reducing incorrect or misleading answers.

3. To Reduce Dependency on Manual Support

To minimize the need for human intervention by automating responses to frequently asked questions, thereby saving time and reducing workload on support staff.

4. To Improve Accessibility of Government Services

To make information about MPOnline services easily accessible to users, especially those who are not familiar with complex procedures or technical platforms.

5. To Integrate Document Processing and OCR

To enable the system to process documents such as PDFs and images using Optical Character Recognition (OCR) and extract useful information for building a knowledge base.

6. To Provide Real-Time Query Resolution

To ensure that users receive instant responses to their queries without delays, improving overall user experience and efficiency.

7. To Implement Grievance Management System

To allow users to submit complaints or unresolved queries and track their status through a structured grievance handling system.

8. To Enable Expert Escalation

To provide a mechanism where complex or unanswered queries can be forwarded to human experts for accurate resolution.

9. To Maintain Chat History and User Sessions

To store user interactions and chat sessions for future reference, allowing users to revisit previous queries and responses.

10. To Provide Analytics and Insights

To analyse user queries, identify frequently asked questions, and improve system performance using an analytics dashboard.

11. To Build a Scalable and Efficient System

To design the system in a way that it can handle a large number of users and queries efficiently without performance degradation.

SCOPE OF THE PROJECT

The project titled “**AI-Powered FAQ Chatbot for MPOnline Services**” focuses on developing an intelligent and automated system that assists users in accessing accurate and relevant information related to various government services available on the MPOnline platform. The scope of this project defines the functionalities, coverage, and limitations of the system.

1. Information Assistance for MPOnline Services

The system is designed to provide detailed information about various MPOnline services such as income certificate, caste certificate, domicile certificate, bill payments, examination forms, and other government-related services. It helps users understand procedures, required documents, eligibility criteria, and application steps.

2. Document-Based Knowledge System

The chatbot operates using a document-based approach, where official documents such as PDFs and images are processed to extract information. This ensures that the responses generated by the system are based on actual data rather than predefined or generic answers.

3. AI-Based Query Handling

The system can handle user queries using artificial intelligence techniques. It uses a Retrieval-Augmented Generation (RAG) approach to retrieve relevant information and generate context-aware responses, making it more accurate and reliable.

4. Public Access Chatbot

The chatbot is accessible through a public landing page, allowing any user to ask questions without the need for login. This feature improves accessibility and ensures that users can quickly get information without any barriers.

5. User Authentication and Personalized Features

Registered users can log in to access additional features such as chat history, saved sessions, grievance submission, and personalized interactions.

6. Grievance Management System

The system allows users to submit complaints or issues if their queries are not resolved. Users can track the status of their grievances, and administrators can manage and resolve them.

efficiently.

7. Expert Escalation Mechanism

In cases where the chatbot is unable to provide a satisfactory answer, the query can be escalated to a human expert. This ensures that complex or critical queries are handled accurately.

8. Admin and Document Management

Administrators have the ability to upload, manage, and update documents that form the knowledge base of the chatbot. This allows the system to stay updated with the latest information.

9. Analytics and Monitoring

The system includes an analytics dashboard that helps administrators monitor user activity, frequently asked questions, and system performance, enabling continuous improvement.

10. System Limitations

While the system provides accurate responses based on available data, it depends on the quality and completeness of the uploaded documents. If relevant information is not present in the knowledge base, the chatbot may not be able to provide an answer and may escalate the query.

11. Future Expandability

The system is designed to be scalable and can be extended to include features such as multi-language support, voice-based interaction, mobile application integration, and integration with real-time government APIs.

DEFINITION OF PROBLEM

The rapid shift towards digital platforms for government services has made systems like MPOnline essential for citizens. However, despite the availability of these services online, many users still face significant difficulties while accessing accurate and complete information. The primary problem lies in the complexity of procedures, lack of clear guidance, and limited availability of real-time support. Users often struggle to understand the correct steps for applying for services such as income certificates, caste certificates, or various registrations. This confusion leads to errors in applications, rejection of forms, and unnecessary delays.

Another major issue is that information is usually scattered across multiple sources, and users are required to search through different pages or documents to find relevant details. This process is not only time-consuming but also confusing, especially for individuals who are not familiar with digital platforms. In rural areas, the problem becomes more serious as users depend heavily on third-party assistance such as cyber cafes, which increases both cost and dependency.

Traditional customer support systems are also not efficient enough to handle a large number of user queries simultaneously. There is often a delay in response, and sometimes users do not receive proper assistance for their specific problems. As a result, many users feel frustrated and are unable to complete their tasks successfully.

Moreover, there is no intelligent system in place that can understand user queries in natural language and provide accurate, context-based answers. Most existing systems rely on static FAQs or manual help, which are not sufficient to address dynamic and specific user questions.

Therefore, there is a need for an automated, intelligent, and scalable solution that can provide instant and reliable assistance to users. The system should be capable of understanding user queries, retrieving relevant information from official documents, and generating meaningful responses. It should also include mechanisms to handle unresolved queries through grievance systems and expert support. This project aims to address these challenges by developing an AI-powered chatbot that simplifies access to MPOnline services and improves the overall user experience.

BENEFITS OF PROJECT

The project titled “**AI-Powered FAQ Chatbot for MPOnline Services**” offers several practical benefits that improve both user experience and service efficiency. One of the major advantages of this system is that it provides instant and accurate responses to user queries. Instead of searching through multiple sources or waiting for manual support, users can directly interact with the chatbot and receive relevant information in real time.

Another important benefit is the reduction in dependency on human assistance. By automating frequently asked questions and common queries, the system minimizes the workload on support staff and allows them to focus on more complex issues. This leads to better resource utilization and improved overall efficiency.

The project also enhances accessibility of government services. Users from both urban and rural areas can easily access information without requiring technical expertise. The simple chat-based interface makes it easy for users to understand procedures, required documents, and application steps without confusion.

The integration of document-based knowledge ensures that the responses provided by the chatbot are reliable and based on actual data. This reduces the chances of incorrect information and builds trust among users. Additionally, the use of artificial intelligence allows the system to understand queries in natural language, making interaction more user-friendly.

Another key benefit is the grievance management system included in the project. Users can submit complaints if their queries are not resolved and track their status, ensuring transparency and accountability. The expert escalation feature further strengthens the system by allowing complex queries to be handled by human experts when required.

The system also provides valuable insights through its analytics feature. It helps in identifying frequently asked questions, common issues faced by users, and areas where improvements are needed. This information can be used to enhance the system over time.

Overall, the project improves efficiency, reduces response time, enhances user satisfaction, and provides a scalable solution for handling large volumes of queries related to MPOnline services.

MAIN MODULE

The project “**AI-Powered FAQ Chatbot for MPOnline Services**” is divided into several important modules. Each module performs a specific function and together they ensure smooth working of the system.

1. User Interface Module

This module provides the interaction layer between the user and the system. It includes:

- Landing page (public access)
- Chatbot interface
- Login and registration pages
- Dashboard for users and admin

It allows users to easily ask questions and navigate through the system without technical knowledge.

2. Authentication Module

This module manages user identity and access control. It includes:

- User registration
- Login (password/OTP based)
- JWT authentication
- Role management (user, admin, expert)

It ensures that only authorized users can access secured features.

3. Chatbot and RAG Processing Module

This is the core module of the system. It performs:

- Query understanding
- Conversion of query into embeddings
- Retrieval of relevant data from vector database
- Response generation using AI model

It ensures accurate and context-based answers using document data.

4. Document Processing Module

This module is responsible for building the knowledge base. It includes:

- Document upload (PDF/Image)
- Text extraction using OCR

- Text chunking
- Embedding generation
- Storage in vector database

This module ensures that the chatbot has reliable data to answer queries.

5. Grievance Management Module

This module allows users to raise complaints. It includes:

- Complaint submission
- Unique grievance ID generation
- Status tracking
- Admin resolution system

It improves transparency and user satisfaction.

6. Expert Escalation Module

This module handles complex queries that the chatbot cannot answer. It includes:

- Automatic escalation (low confidence answers)
- Manual escalation by user
- Query resolution tracking

It ensures no query remains unanswered.

7. Analytics Module

This module provides insights into system usage. It includes:

- Frequently asked questions
- Failed queries
- User activity tracking
- Feedback analysis

It helps in improving system performance.

8. Database and Storage Module

This module manages all system data. It includes:

- User data storage
- Chat history and sessions
- Grievance and feedback data
- Vector storage (FAISS)

It ensures efficient data management and fast retrieval.

TECHNICAL OVERVIEW

The project “**AI-Powered FAQ Chatbot for MPOnline Services**” is developed using a combination of modern web technologies and artificial intelligence tools. The system follows a full-stack architecture where both frontend and backend technologies work together to provide a seamless user experience. The main technologies used in this project include HTML, CSS, JavaScript, React.js, Python, FastAPI, PostgreSQL, and vector database techniques.

Frontend Technologies

The frontend of the system is developed using **HTML, CSS, and JavaScript**, along with the React.js library.

- **HTML (HyperText Markup Language)** is used to design the basic structure of web pages such as the landing page, login page, and chatbot interface.
- **CSS (Cascading Style Sheets)** is used to style the application, including layout design, colors, fonts, and responsive behavior.
- **JavaScript** is used to make the web pages interactive, handle events, and communicate with backend APIs.

Additionally, **React.js** is used to build a dynamic and responsive user interface. It allows the application to be divided into reusable components such as chat window, message bubbles, dashboard, and forms. React improves performance and provides a smooth user experience.

Backend Technologies

The backend of the system is developed using **Python** with the **FastAPI framework**.

- **Python** is chosen because of its simplicity and strong support for artificial intelligence and data processing.
- **FastAPI** is a modern web framework used for building APIs. It handles requests from the frontend, processes user queries, manages authentication, and connects different parts of the system.

The backend is responsible for core functionalities such as:

- Handling user login and registration
- Processing chatbot queries
- Managing documents and OCR
- Handling grievances and expert responses

Database System

The project uses **PostgreSQL** as the relational database.

It stores structured data such as:

- User information
- Chat sessions and messages
- Grievance details
- Feedback and analytics data

PostgreSQL is reliable, secure, and capable of handling large amounts of data efficiently.

Vector Database and AI Processing

The system uses a vector database based on **FAISS (Facebook AI Similarity Search)** for storing and retrieving document embeddings.

When documents are uploaded, their content is converted into numerical vectors and stored. During query processing, the system searches for the most relevant data using similarity matching.

The project also integrates AI models such as OpenAI or Gemini APIs for generating intelligent responses. The system follows a **Retrieval-Augmented Generation (RAG)** approach, where relevant data is first retrieved and then used to generate accurate answers.

API Communication

The frontend and backend communicate through REST APIs. These APIs handle different operations such as sending user queries, receiving chatbot responses, uploading documents, and managing user data. This ensures a modular and scalable system design.

System Integration

All technologies are integrated in a structured manner where:

- The frontend sends user requests
- The backend processes the request
- The database stores and retrieves data
- The AI model generates the final response

This integration ensures smooth workflow, fast response time, and accurate results.

SYSTEM CONFIGURATION

The system configuration for the project “**AI-Powered FAQ Chatbot for MPOnline Services**” defines the hardware and software requirements needed for the proper development, execution, and deployment of the application. The system is designed in such a way that it can run efficiently on standard computer systems without requiring high-end resources.

Hardware Requirements

The hardware requirements for this project are minimal and easily available. A standard personal computer or laptop is sufficient for development and execution.

- **Processor:** Intel Core i3 or higher
- **RAM:** Minimum 4 GB (8 GB recommended for better performance)
- **Storage:** Minimum 256 GB hard disk or SSD
- **Input Devices:** Keyboard and mouse
- **Internet Connection:** Required for API integration and online services

These specifications ensure that the system can handle backend processing, frontend rendering, and database operations smoothly.

Software Requirements

The software requirements include the operating system, programming tools, frameworks, and supporting technologies used in the project.

- **Operating System:** Windows, Linux, or macOS
- **Programming Language:** Python (version 3.9 or above)
- **Frontend Technologies:** HTML, CSS, JavaScript, React.js
- **Backend Framework:** FastAPI
- **Database:** PostgreSQL
- **Vector Database:** FAISS
- **OCR Tool:** Tesseract OCR
- **API Integration:** OpenAI API or Gemini API
- **Package Manager:** npm (for frontend), pip (for backend)

Development Tools

The following tools are used during the development phase:

- **Code Editor/IDE:** Visual Studio Code or PyCharm
- **Version Control:** Git and GitHub
- **API Testing Tool:** Postman or browser-based testing
- **Database Management Tool:** pgAdmin or DBeaver

These tools help in writing, testing, and managing the project efficiently.

Deployment Requirements

For deployment, the system can be hosted on cloud platforms.

- **Backend Hosting:** Railway, Render, or VPS server
- **Frontend Hosting:** Vercel or Netlify
- **Database Hosting:** Neon, Supabase, or PostgreSQL server
- **Storage:** Local storage or cloud storage for documents

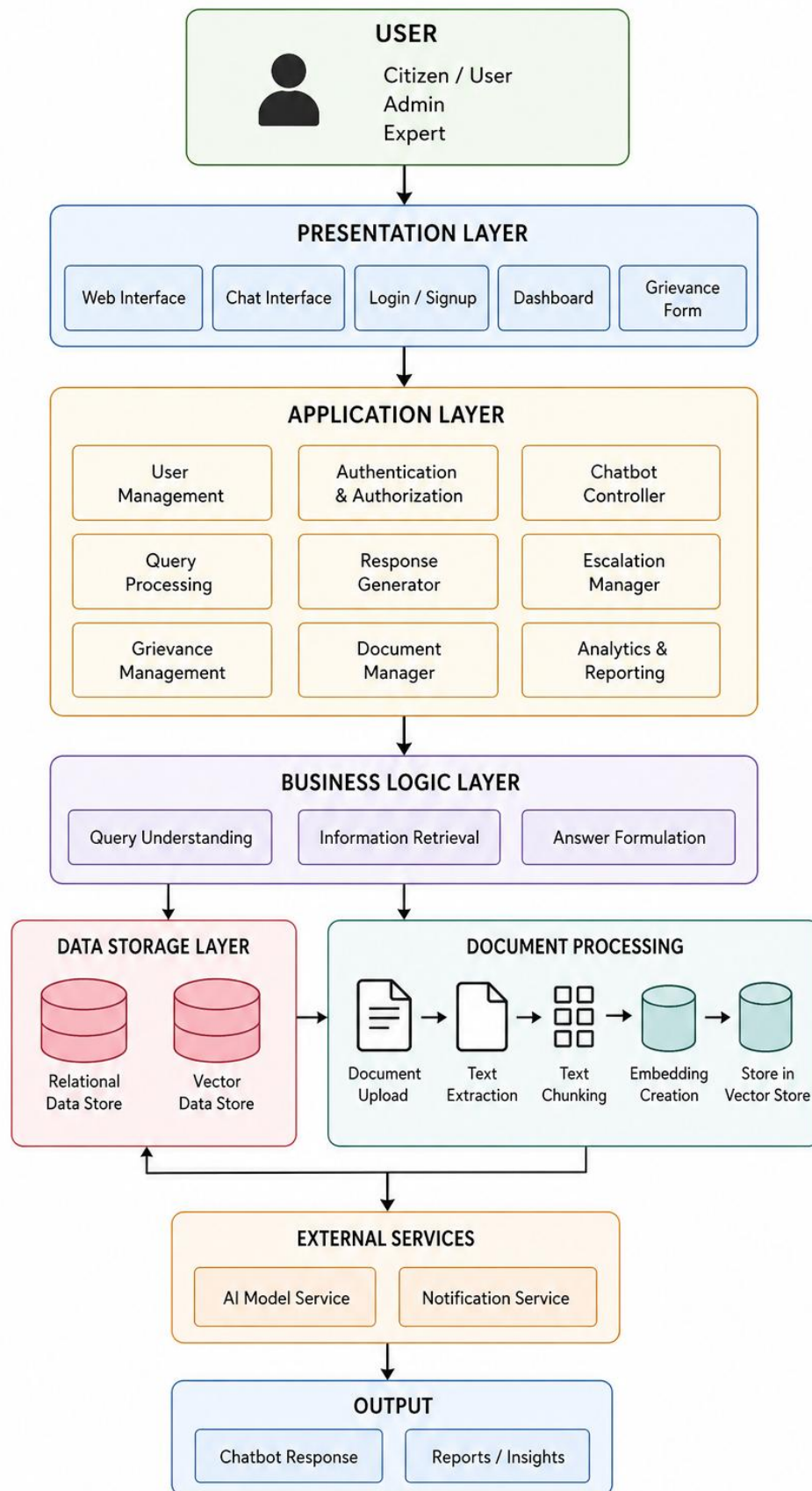
System Environment

The system uses environment variables for configuration, including database connection, API keys, and application settings. This ensures security and flexibility during development and deployment.

Performance Considerations

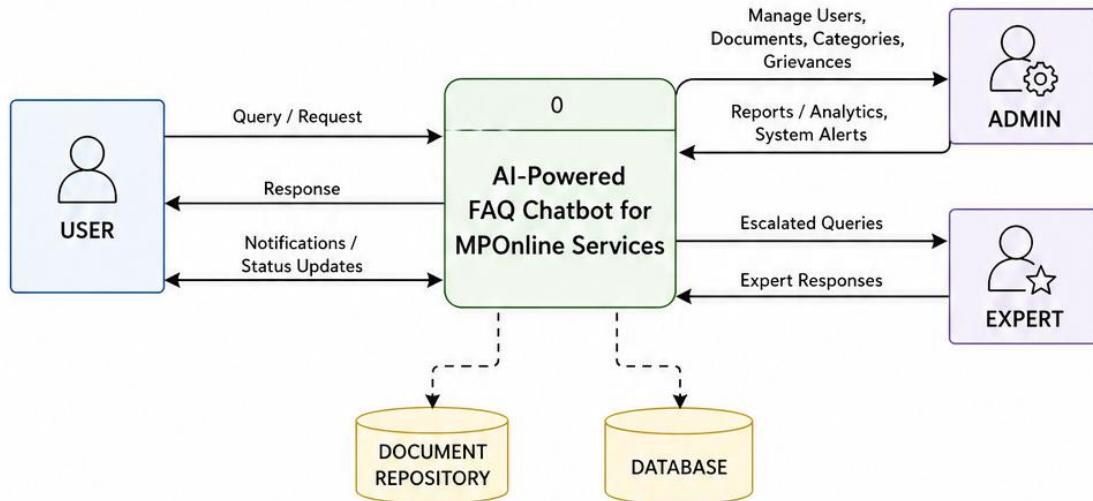
The system is optimized to handle multiple user requests efficiently. It uses asynchronous processing in the backend and optimized database queries to ensure fast response time. The use of vector database improves retrieval speed, and caching techniques can be applied to further enhance performance.

BLOCK DIAGRAM

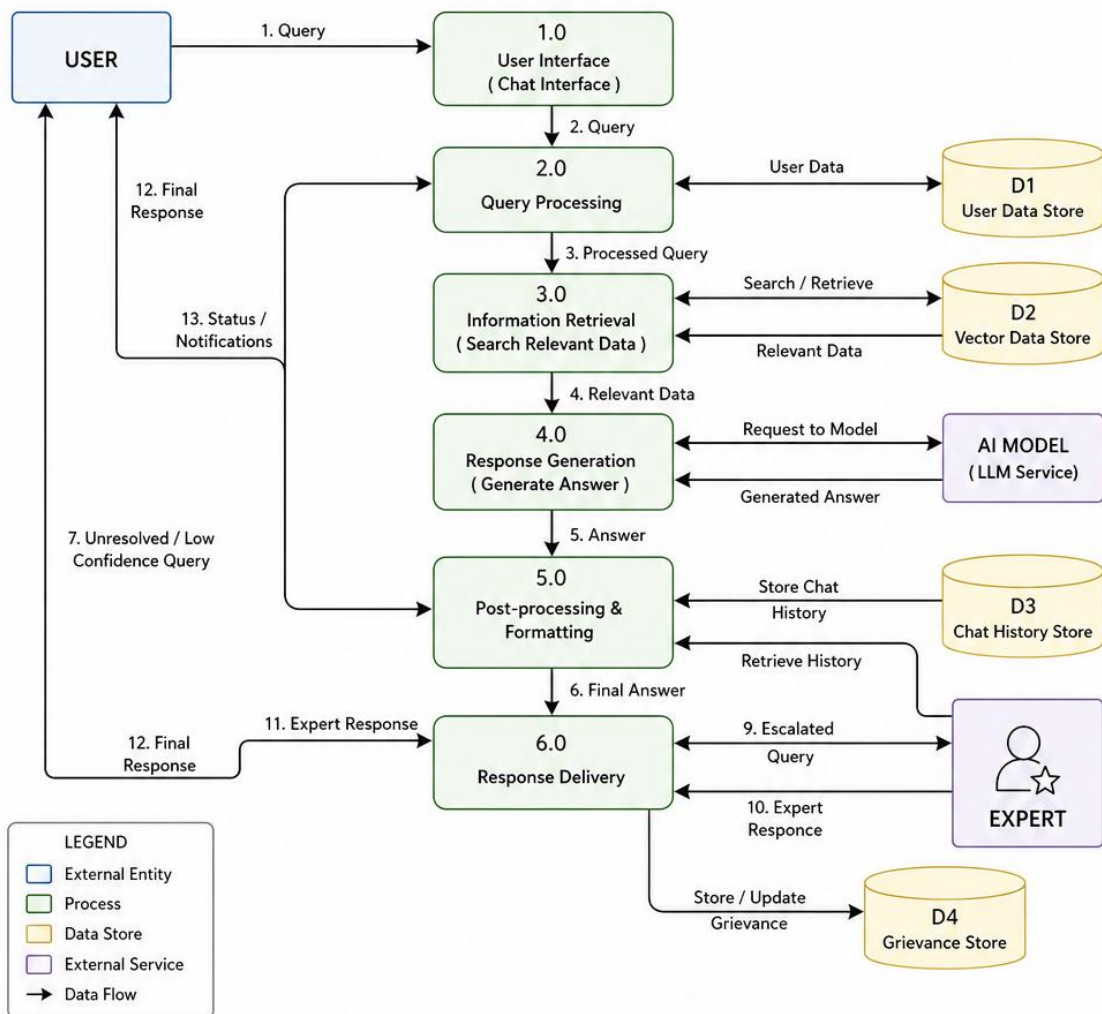


DFD DIAGRAM

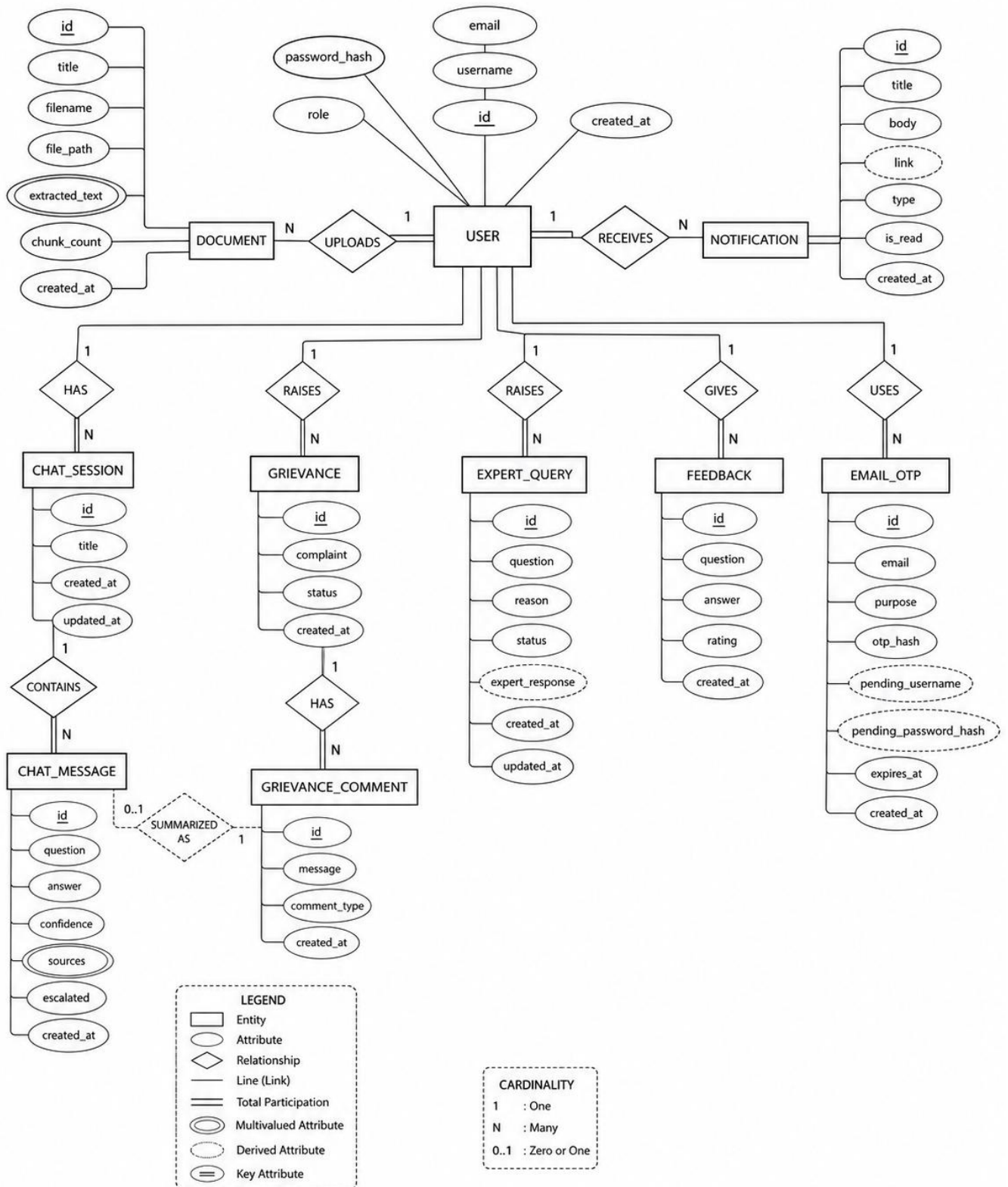
DFD LEVEL 0 (CONTEXT DIAGRAM)



DFD LEVEL 1



ER DIAGRAM



OUTPUT SCREEN

Landing Page



Login and Signup Page

AI + RAG

MPOnline FAQ Chatbot

Sign in with your email using either password or OTP. New users verify their email with an OTP before the account is created.

Login With Password

Login With OTP

Login

Don't have an account? **Register**

AI + RAG

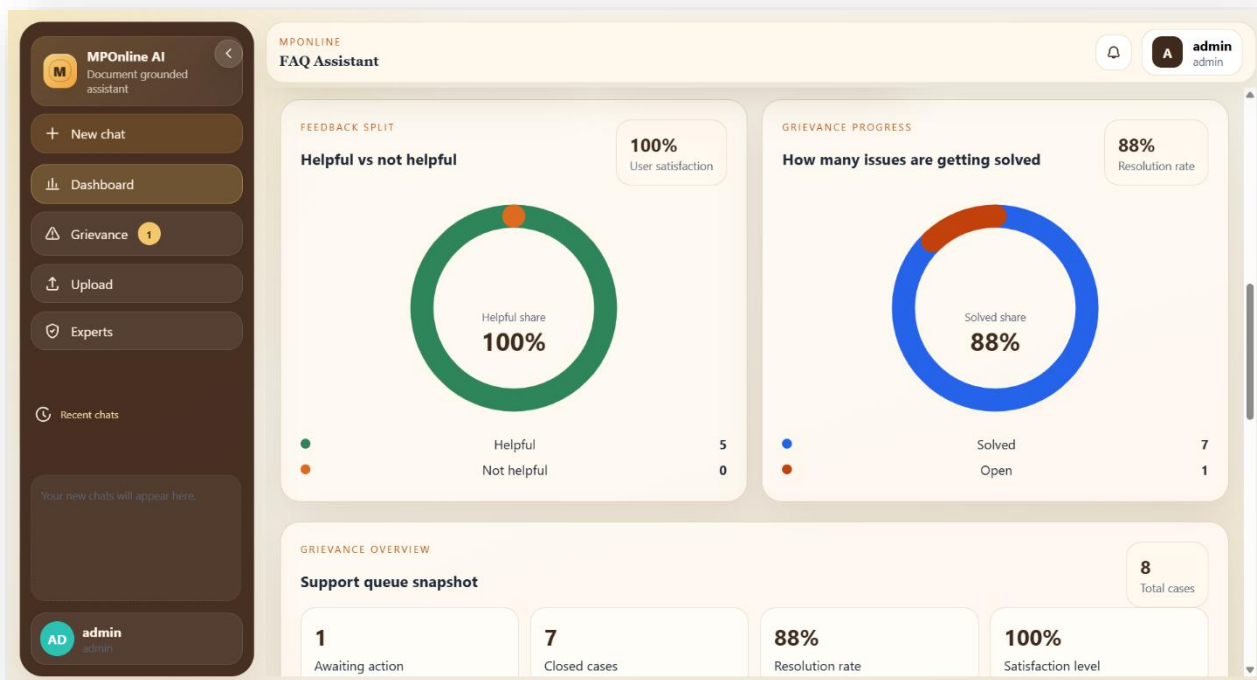
MPOnline FAQ Chatbot

Sign in with your email using either password or OTP. New users verify their email with an OTP before the account is created.

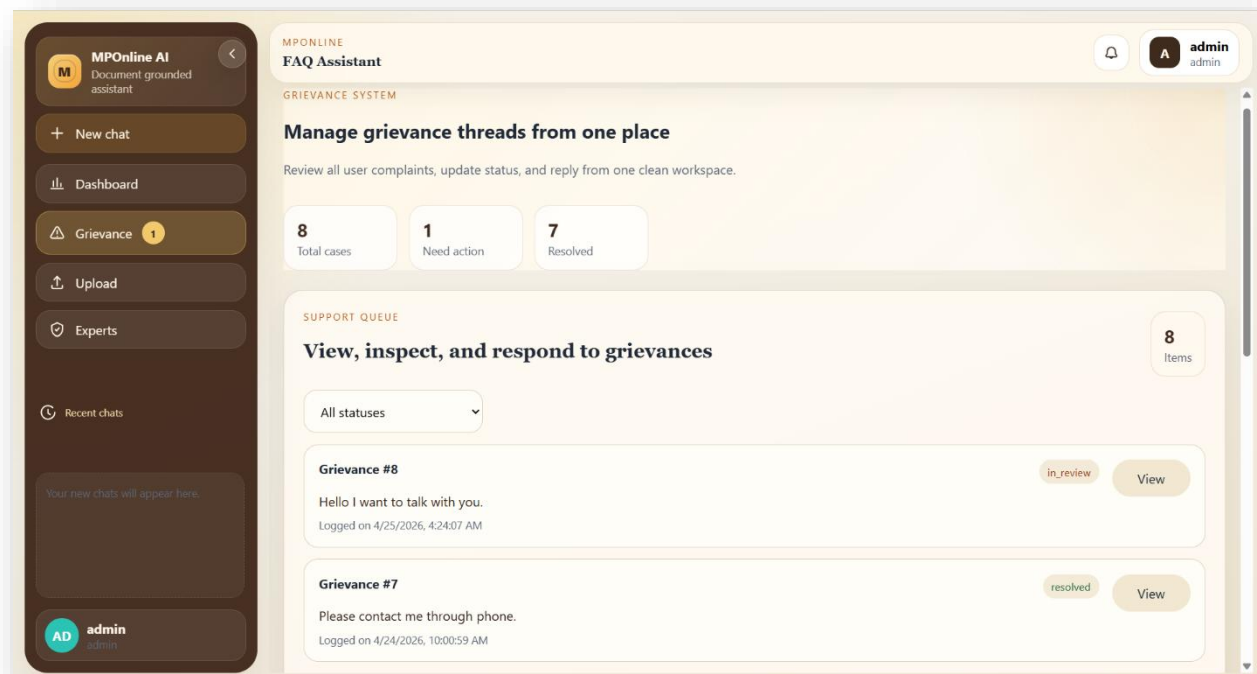
Send Registration OTP

Already have an account? **Login**

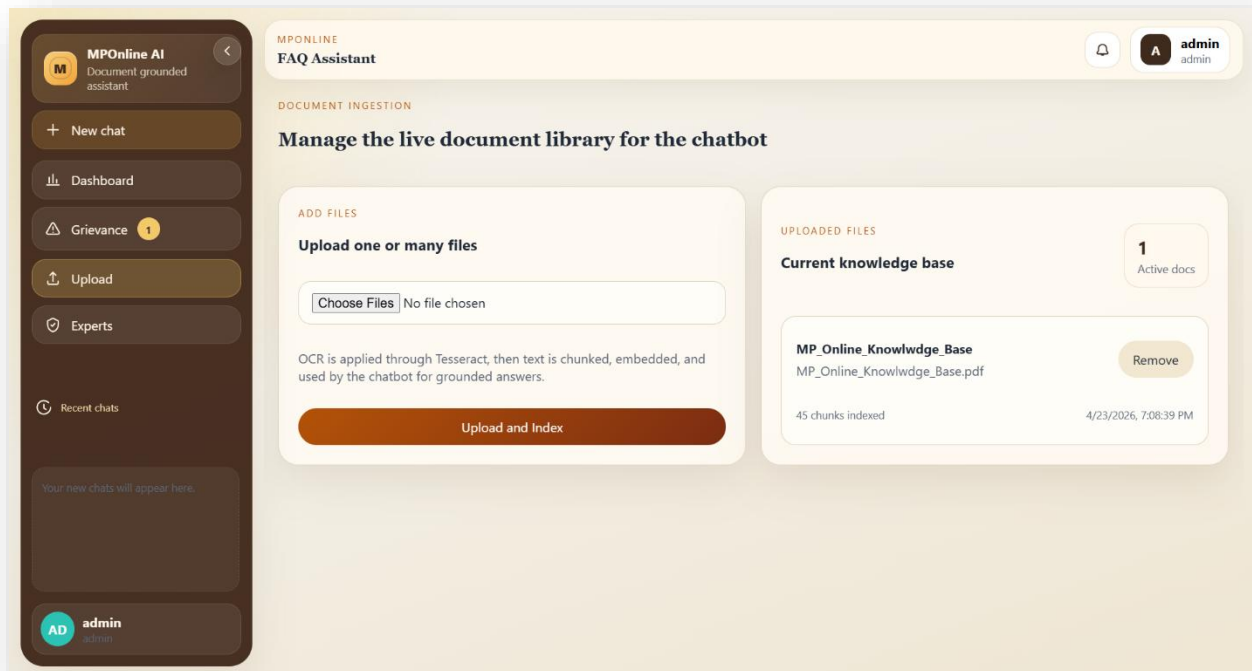
Admin Dashboard



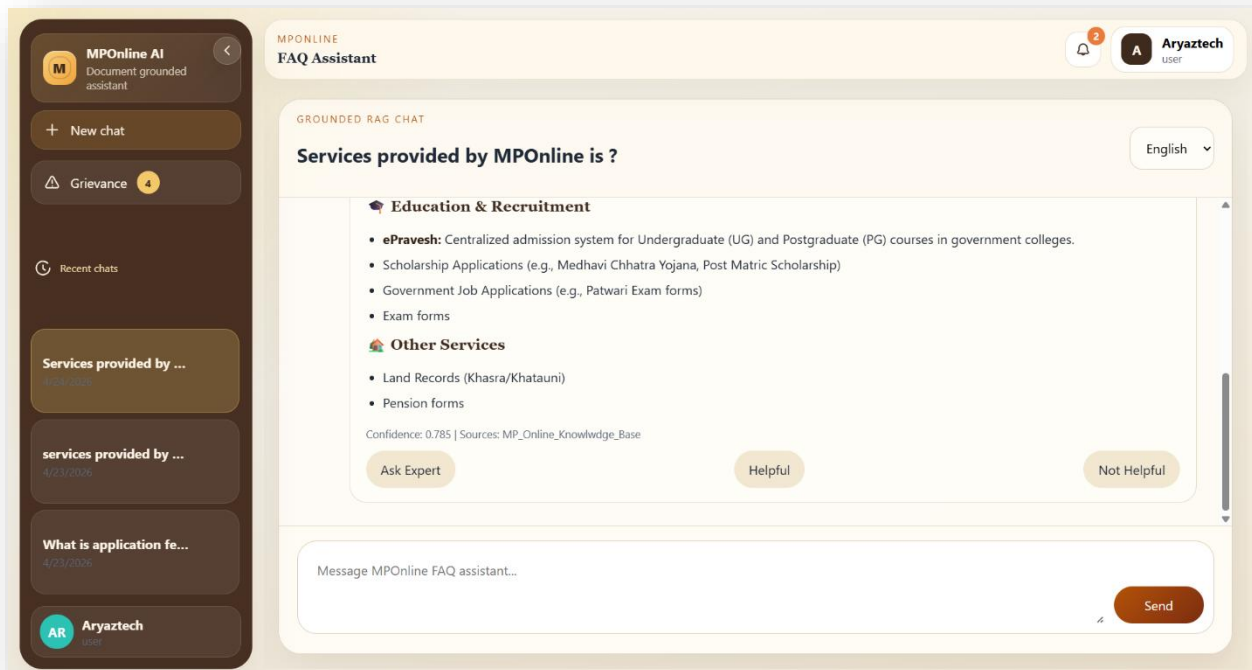
Admin Grievance Page



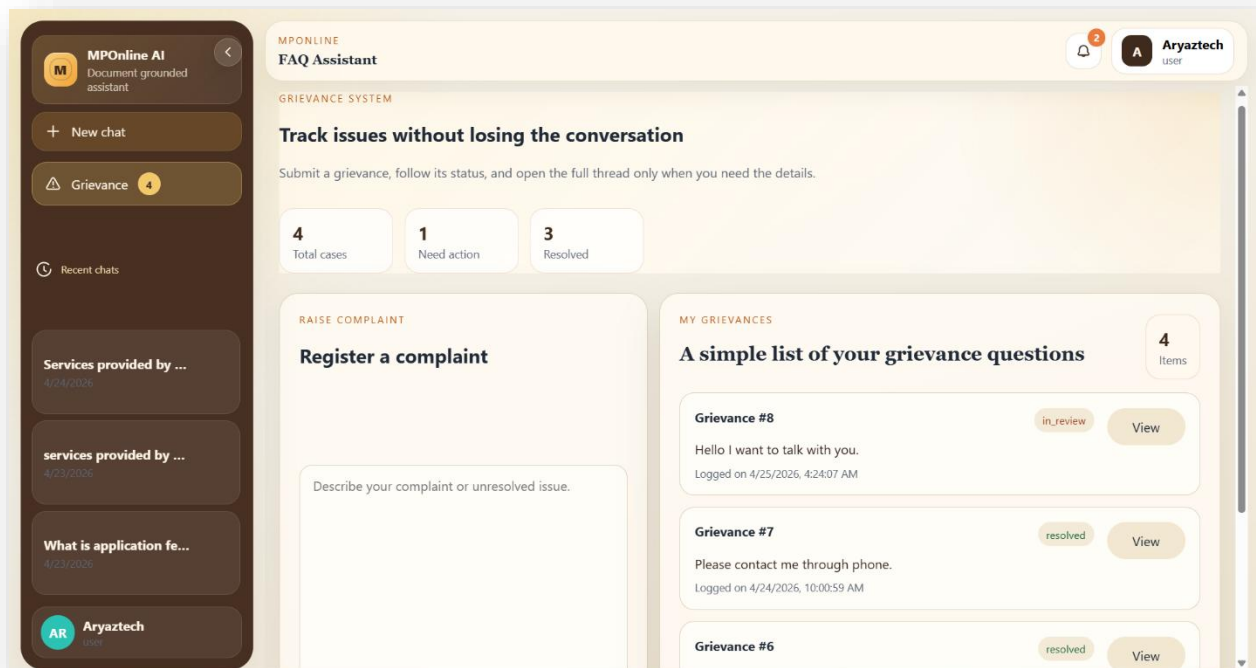
Knowledge Base Upload Page



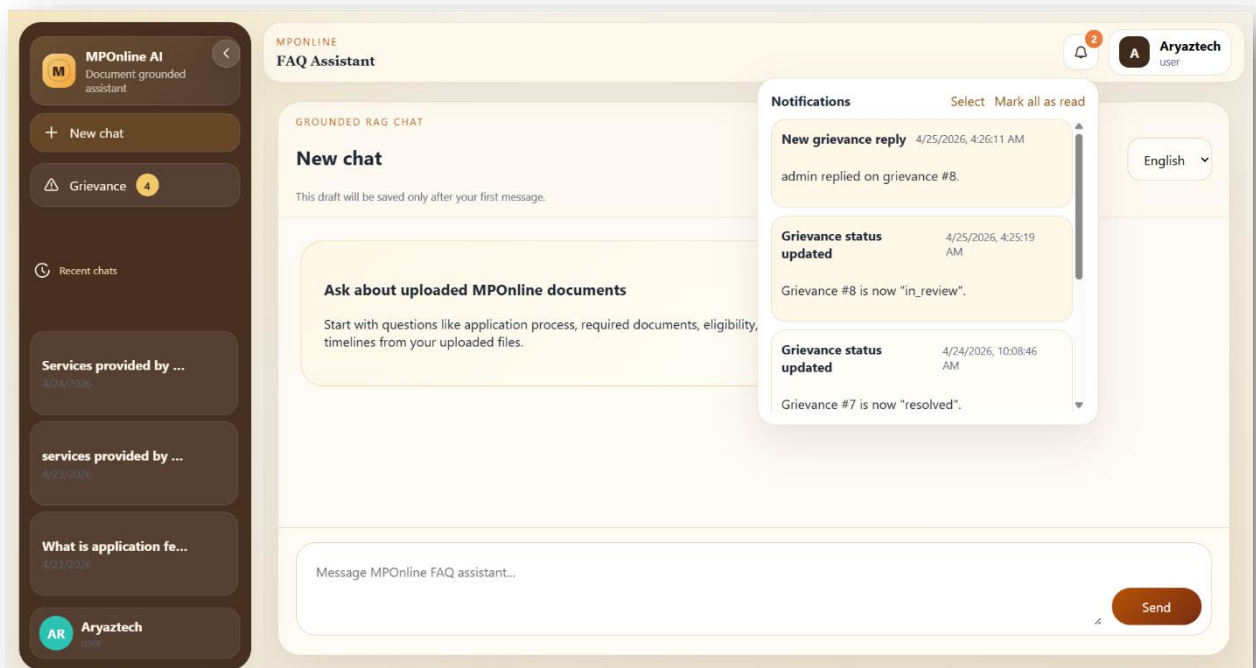
User Chat Dashboard



User Grievance Panel



New Chat window with Notification window



PROJECT EXPLANATION

Overview of the System

The project titled “AI-Powered FAQ Chatbot for MPOnline Services” is developed to provide an intelligent, automated, and user-friendly solution for handling queries related to government services. The system is designed to reduce the complexity faced by users while accessing MPOnline services and to provide accurate information in real time. It combines web technologies with artificial intelligence to create a smart assistance platform that can understand user queries and respond in a meaningful way. The system is scalable and capable of handling multiple users simultaneously, making it suitable for real-world deployment.

Over User Interaction Flow

The system begins with the user interface, which acts as the entry point for all interactions. Users can access the application through a web browser. The landing page is publicly accessible, which allows users to ask questions without the need for login. This ensures that basic information is easily available to everyone. The chatbot is integrated into the interface and allows users to type their queries in natural language. For additional features such as saving chat history, accessing previous sessions, submitting grievances, and receiving notifications, users are required to register and log in. The interface is designed to be simple and easy to use so that even users with limited technical knowledge can interact with the system without difficulty.

Query Processing Mechanism

When a user submits a query, it is sent to the backend server through an API request. The backend first performs validation of the input to ensure that the query is not empty and is within the allowed length. After validation, the system processes the query to understand its intent. The query is then converted into a numerical representation using embedding techniques. This conversion allows the system to compare the user query with stored data efficiently. The processed query is then passed to the retrieval system for further processing.

Document Processing and Knowledge Base

The system builds its knowledge base from documents uploaded by administrators. These documents may include PDFs, text files, or scanned images containing information related to MPOnline services. When a document is uploaded, the system uses Optical Character Recognition to extract text from the file. This ensures that even scanned documents can be

used as a data source. The extracted text is then divided into smaller parts called chunks. Chunking improves the efficiency of the retrieval process and allows more precise matching of user queries. Each chunk is then converted into an embedding and stored in a vector database. This database allows fast and accurate similarity search during query processing.

Retrieval-Augmented Generation

The system follows a Retrieval-Augmented Generation approach, which is a combination of information retrieval and response generation. In the first step, the system retrieves the most relevant chunks of data from the vector database based on the user query. In the second step, the retrieved data is provided as context to the AI model, which generates a final response. This approach ensures that the answers are based on actual data rather than general knowledge, which significantly improves accuracy and reliability. It also reduces the chances of incorrect or misleading responses.

Response Generation and Delivery

Once the relevant data is retrieved, it is sent along with the user query to the AI model. The model processes the information and generates a response that is clear, structured, and context-aware. The generated response is then sent back to the backend server and finally delivered to the frontend, where it is displayed in the chat interface. The system ensures that the response is easy to understand and directly addresses the user's query. This process happens in real time, providing instant assistance to the user.

Chat History and Session Management

The system maintains a record of all user interactions. Each query and its corresponding response are stored in the database as part of chat history. The system also organizes conversations into chat sessions, which helps in grouping related queries. Users can access their previous conversations, which improves usability and allows them to refer back to earlier responses. This feature is particularly useful for users who need to revisit instructions or track their previous queries.

Expert Escalation Mechanism

In cases where the system is unable to provide a confident or accurate answer, the query is flagged for escalation. The user is given an option to send the query to a human expert. Once escalated, the query is forwarded to the expert panel, where a knowledgeable person reviews the question and provides a detailed response. This response is then delivered back to the user.

through the system. The escalation mechanism ensures that even complex or uncommon queries are handled properly and that users always receive a solution.

Grievance Management System

The system includes a grievance management module to handle user complaints. If a user is not satisfied with the response or faces any issue, they can submit a complaint through the system. Each grievance is assigned a unique identifier, which allows users to track its progress. Administrators and experts can view all grievances, update their status, and provide solutions. This feature ensures transparency and accountability, and it improves user trust in the system.

Notification System

The system provides a notification feature to keep users informed about important updates. Users receive notifications regarding grievance status changes, expert responses, and other system-related alerts. This ensures that users are always aware of the progress of their queries and do not need to manually check for updates.

Analytics and Monitoring

The system includes an analytics module that helps administrators monitor system performance and user behavior. It collects data such as total number of queries, frequently asked questions, failed responses, and user feedback. This information is useful for identifying areas where the system can be improved. By analyzing this data, administrators can update the knowledge base and enhance the overall efficiency of the chatbot.

Security and Authentication

Security is an important aspect of the system. The application uses authentication mechanisms to ensure that only authorized users can access certain features. User data is stored securely, and proper access control is maintained. Role-based access is implemented to differentiate between normal users, administrators, and experts. This ensures that each user has access only to the features relevant to their role.

Overall System Workflow

The complete workflow of the system can be described as follows. The user enters a query through the interface. The query is sent to the backend, where it is processed and converted into an embedding. The system retrieves relevant information from the vector database and sends it to the AI model. The model generates a response, which is then displayed to the user. If the response is not satisfactory, the query can be escalated to an expert or handled through

the grievance system. All interactions are stored in the database for future reference.

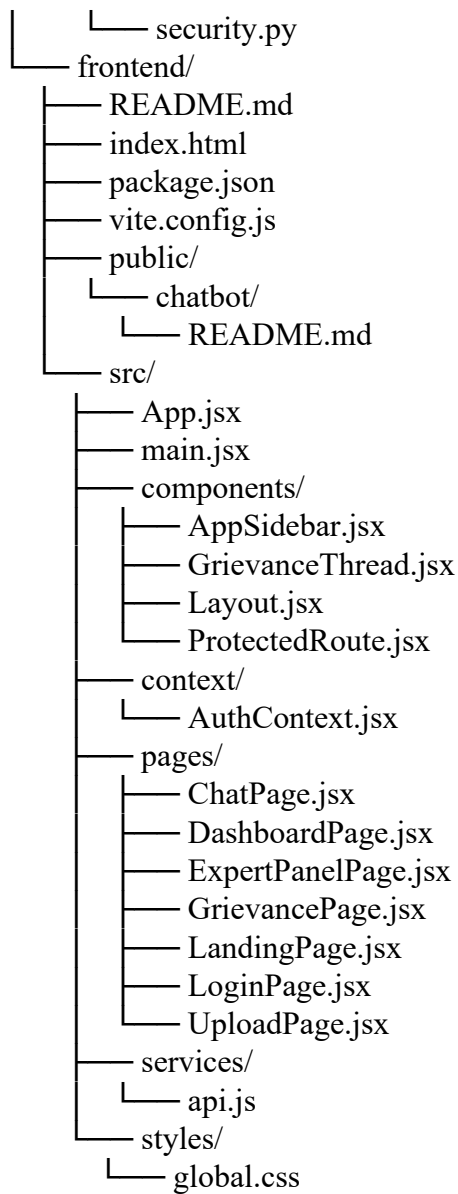
Conclusion of Working

The project provides a complete and efficient solution for handling user queries related to MPOnline services. It simplifies complex processes, reduces dependency on manual support, and improves accessibility for users. By combining artificial intelligence with a structured system design, the application delivers accurate, fast, and reliable responses. The system is scalable and can be further enhanced with additional features, making it a practical solution for real-world applications.

CODING

Directory structure

```
├── ayushcodespy-faq-chatbot-mponline/
│   ├── README.md
│   ├── API_DOCUMENTATION.md
│   ├── requirements.txt
│   ├── SYSTEM_FLOW_EXPLANATION.md
│   ├── .env.example
│   └── app/
│       ├── __init__.py
│       ├── config.py
│       ├── database.py
│       ├── dependencies.py
│       ├── main.py
│       ├── rag_pipeline.py
│       ├── models/
│       │   ├── __init__.py
│       │   ├── entities.py
│       │   └── schemas.py
│       ├── routes/
│       │   ├── __init__.py
│       │   ├── analytics.py
│       │   ├── auth.py
│       │   ├── chat.py
│       │   ├── documents.py
│       │   ├── expert.py
│       │   ├── feedback.py
│       │   ├── grievance.py
│       │   └── notifications.py
│       ├── services/
│       │   ├── __init__.py
│       │   ├── ai_service.py
│       │   ├── analytics_service.py
│       │   ├── auth_service.py
│       │   ├── chat_service.py
│       │   ├── document_service.py
│       │   ├── email_service.py
│       │   ├── notification_service.py
│       │   ├── ocr_service.py
│       │   └── vector_store.py
│       ├── uploads/
│       │   └── .gitkeep
│       └── utils/
│           ├── __init__.py
│           └── chunking.py
```



1. Main FastAPI Entry Point (main.py)

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy import inspect, text
from app.config import get_settings
from app.database import Base, SessionLocal, engine
from app.routes.analytics import router as analytics_router
from app.routes.auth import router as auth_router
from app.routes.chat import router as chat_router
from app.routes.documents import router as documents_router
from app.routes.expert import router as expert_router
from app.routes.feedback import router as feedback_router
from app.routes.grievance import router as grievance_router
from app.routes.notifications import router as notifications_router
from app.services.auth_service import ensure_bootstrap_admin

settings = get_settings()

def ensure_runtime_columns() -> None:
    inspector = inspect(engine)
    document_columns = {column["name"] for column in
inspector.get_columns("documents")}
    with engine.begin() as connection:
        if "title" not in document_columns:
            connection.execute(text("ALTER TABLE documents ADD COLUMN title
VARCHAR(255)"))
            connection.execute(text("UPDATE documents SET title = filename WHERE
title IS NULL"))

@asynccontextmanager
async def lifespan(_: FastAPI):
    Base.metadata.create_all(bind=engine)
    ensure_runtime_columns()
    db = SessionLocal()
    try:
        ensure_bootstrap_admin(
            db,
            settings.bootstrap_admin_username,
            settings.bootstrap_admin_email,
            settings.bootstrap_admin_password,
        )
```

```
        finally:
            db.close()
        yield

app = FastAPI(
    title=settings.app_name,
    debug=settings.debug,
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.allowed_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/")
def healthcheck() -> dict:
    return {
        "name": settings.app_name,
        "status": "ok",
        "docs": "/docs",
    }
```

Explanation:

- This file is the **main entry point** of the FastAPI backend application.
- It loads configuration settings like app name, debug mode, and API prefix.
- The **lifespan function** runs at startup and:
 - Creates database tables
 - Initializes default admin user
- FastAPI app is initialized with:
 - Title
 - Version
 - Debug mode
- **CORS middleware** is added to allow communication between frontend and backend.

- Multiple routers are included for different modules:
 - Authentication
 - Chat system
 - Document handling
 - Grievance management
 - Expert system
 - Notifications
 - Feedback
 - Analytics
- Each router handles a specific functionality, making the system **modular and scalable**.
- A **health check API** is defined to verify that the server is running properly.

2. Authentication Logic (auth_service.py)

```
def _generate_otp() -> str:
    return f"{randbelow(900000) + 100000}"

def _delete_existing_otps(db: Session, email: str, purpose: str) -> None:
    db.query(EmailOTP).filter(EmailOTP.email == email, EmailOTP.purpose ==
purpose).delete()

def _create_otp_record(
    db: Session,
    *,
    email: str,
    purpose: str,
    otp: str,
    pending_username: str | None = None,
    pending_password_hash: str | None = None,
) -> None:
    settings = get_settings()
    _delete_existing_otps(db, email, purpose)
    db.add(
        EmailOTP(
            email=email,
            purpose=purpose,
            otp_hash=hash_password(otp),
            pending_username=pending_username,
            pending_password_hash=pending_password_hash,
            expires_at=datetime.now(timezone.utc) +
timedelta(minutes=settings.otp_expiry_minutes),
        )
    )
    db.commit()

def _get_valid_otp(db: Session, email: str, purpose: str, otp: str) ->
EmailOTP:
    record = (
        db.query(EmailOTP)
        .filter(EmailOTP.email == email, EmailOTP.purpose == purpose)
        .order_by(EmailOTP.created_at.desc())
        .first()
    )
    if not record:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="OTP
not found. Please request a new one.")
    if record.expires_at.replace(tzinfo=timezone.utc) <
datetime.now(timezone.utc):
```

```
        db.delete(record)
        db.commit()
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="OTP has expired. Please request a new one.")
        if not verify_password(otp, record.otp_hash):
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid OTP.")
        return record

def register_user(db: Session, payload: UserCreate) -> User:
    existing = (
        db.query(User)
        .filter((User.username == payload.username) | (User.email ==
payload.email))
        .first()
    )
    if existing:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail="User
already exists")

    user = User(
        username=payload.username,
        email=payload.email,
        password_hash=hash_password(payload.password),
        role=UserRole.USER,
    )
    db.add(user)
    db.commit()
    db.refresh(user)
    return user

def send_registration_otp(db: Session, payload: RegisterRequest) -> None:
    existing = (
        db.query(User)
        .filter((User.username == payload.username) | (User.email ==
payload.email))
        .first()
    )
    if existing:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail="User
already exists")

    otp = _generate_otp()
    settings = get_settings()
    _create_otp_record(
        db,
        email=payload.email,
```

```
        purpose="register",
        otp=otp,
        pending_username=payload.username,
        pending_password_hash=hash_password(payload.password),
    )
    send_registration_otp_email(payload.email, payload.username, otp,
settings.otp_expiry_minutes)

def verify_registration_otp(db: Session, payload: RegisterVerifyRequest) ->
tuple[str, User]:
    record = _get_valid_otp(db, payload.email, "register", payload.otp)
    if not record.pending_username or not record.pending_password_hash:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Registration data is incomplete. Please register again.")

    existing = (
        db.query(User)
        .filter((User.username == record.pending_username) | (User.email ==
record.email))
        .first()
    )
    if existing:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail="User
already exists")

    user = User(
        username=record.pending_username,
        email=record.email,
        password_hash=record.pending_password_hash,
        role=UserRole.USER,
    )
    db.add(user)
    db.commit()
    db.refresh(user)
    db.delete(record)
    db.commit()
    return create_access_token(str(user.id)), user

def authenticate_user_with_password(db: Session, payload: UserLogin) ->
tuple[str, User]:
    user = db.query(User).filter(User.email == payload.email).first()
    if not user or not verify_password(payload.password, user.password_hash):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid email or password",
        )
```



```
token = create_access_token(str(user.id))  
return token, user
```

Explanation:

- This module handles **user authentication and authorization** in the system.
- It supports both **password-based login** and **OTP-based login/registration**.
- OTP (One-Time Password) is generated using a secure random function for verification.
- OTP is stored in the database with:
 - Expiry time
 - Hashed value (for security)
- Before creating a new OTP, existing OTP records for the same user are deleted to avoid conflicts.
- During verification:
 - System checks OTP validity
 - Checks expiry time
 - Matches hashed OTP
- If OTP is valid, user registration or login is completed successfully.
- Passwords are stored securely using **hashing techniques**, not in plain text.
- JWT (JSON Web Token) is generated after successful authentication for secure session handling.
- The module prevents duplicate users by checking existing username or email.
- Email service is integrated to send OTP for:
 - Registration
 - Login
- It includes a function to create a **default admin user** during system startup.
- Error handling is implemented using HTTP exceptions for cases like:
 - Invalid credentials
 - Expired OTP
 - User not found

3. Chat API Endpoints

Public Chat API

```
@router.post("/public/chat")
def public_chat(payload: ChatRequest):
    result = answer_public_question(payload.question, payload.language)
    return result
```

Main Chat API

```
@router.post("/chat")
def chat(
    payload: ChatRequest,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    result = answer_question(
        db,
        current_user,
        payload.question,
        payload.language,
        payload.session_id,
    )
    return result
```

Chat Session API

```
@router.post("/chat/sessions")
def create_session(
    payload: ChatSessionCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
):
    session = create_chat_session(db, current_user, payload.title)
    return session
```

```
@router.delete("/chat/sessions/{session_id}")
def delete_session(
    session_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
) -> dict:
    delete_chat_session(db, current_user, session_id)
    return {"message": "Chat session deleted successfully."}
```

Explanation:

- This module defines **API endpoints for chatbot functionality**.
- The **public chat API** allows users to ask questions without login.
- The **main chat API** handles authenticated users and:
 - Receives user query
 - Passes it to service layer
 - Returns AI-generated response
- Dependency injection is used:
 - Database connection
 - Current logged-in user
- The **chat session API** is used to:
 - Create new chat sessions
 - Organize conversations
- The module follows a **clean architecture**:
 - Routes → Service layer → Database
- Data is returned in structured format using response models.
- Additional APIs (history, reset, delete) are used for managing chat data.

4. Document Processing (ocr_service.py)

Image OCR

```
def extract_text_from_image(file_path: str) -> str:

    image = Image.open(file_path)
    text = pytesseract.image_to_string(image, lang="eng+hin")
    return clean_text(text)
```

PDF OCR

```
def extract_text_from_pdf(file_path: str) -> str:
    import fitz
    document = fitz.open(file_path)
    pages = []

    for page in document:
        text = page.get_text("text")
        pages.append(text)

    return clean_text(" ".join(pages))
```

Main Extract Function

```
def extract_text(file_path: str) -> str:
    if file_path.endswith(".pdf"):
        return extract_text_from_pdf(file_path)

    return extract_text_from_image(file_path)
```

Explanation:

- This module is used for **extracting text from documents**.
- It supports both:
 - Images (JPG, PNG)
 - PDF files
- The **image OCR function**:
 - Opens image using PIL
 - Uses Tesseract OCR to extract text
 - Supports English and Hindi languages
- The **PDF extraction function**:
 - Reads each page of PDF
 - Extracts text content
 - Combines all pages into one text
- If PDF contains scanned images, OCR is applied for extraction.
- The **main function** automatically detects file type:
 - PDF → uses PDF extraction
 - Image → uses OCR
- Extracted text is cleaned before further processing
- This module is important for building the **knowledge base of the chatbot**.

5. Database Model (entities.py)

User Entity

```
class User(Base):
    __tablename__ = "users"

    id = mapped_column(Integer, primary_key=True)
    username = mapped_column(String(50), unique=True)
    email = mapped_column(String(255), unique=True)
    password_hash = mapped_column(String(255))
    role = mapped_column(String(20))
    created_at = mapped_column(DateTime)
```

Chat Session Entity

```
class ChatSession(Base):
    __tablename__ = "chat_sessions"

    id = mapped_column(Integer, primary_key=True)
    user_id = mapped_column(ForeignKey("users.id"))
    title = mapped_column(String(255))
    created_at = mapped_column(DateTime)
```

Chat Message Entity

```
class ChatMessage(Base):
    __tablename__ = "chat_messages"

    id = mapped_column(Integer, primary_key=True)
    session_id = mapped_column(ForeignKey("chat_sessions.id"))
    question = mapped_column(Text)
    answer = mapped_column(Text)
    confidence = mapped_column(Float)
```

Grievance Entity

```
class Grievance(Base):
    __tablename__ = "grievances"

    id = mapped_column(Integer, primary_key=True)
    user_id = mapped_column(ForeignKey("users.id"))
    complaint = mapped_column(Text)
    status = mapped_column(String(50))
```

Document Entity

```
class Document(Base):  
    __tablename__ = "documents"  
    id = mapped_column(Integer, primary_key=True)  
    filename = mapped_column(String(255))  
    extracted_text = mapped_column(Text)  
    chunk_count = mapped_column(Integer)
```

Explanation:

This file defines the **database models (tables)** using SQLAlchemy ORM.

- Each class represents a **table in the database**.
- The **User table** stores:
 - User details
 - Login credentials
 - Role (user, admin, expert)
- The **ChatSession table** is used to:
 - Organize conversations
 - Group multiple messages
- The **ChatMessage table** stores:
 - User questions
 - AI-generated answers
 - Confidence score
- The **Grievance table** handles:
 - User complaints
 - Status tracking
- The **Document table** stores:
 - Uploaded files
 - Extracted text
 - Chunk data for AI
- Relationships (ForeignKey) are used to:
 - Connect users with chats
 - Connect sessions with messages
- This structure ensures:
 - Proper data organization
 - Fast retrieval

6. React API Call (api.js)

API Configuration

```
import axios from "axios";

const API_BASE_URL = "http://localhost:8000/api";

export const api = axios.create({
  baseURL: API_BASE_URL,
});
```

Authorization Token Handling

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("auth_token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Session Storage

```
export const authStorage = {
  setSession: (token, user) => {
    localStorage.setItem("auth_token", token);
    localStorage.setItem("auth_user", JSON.stringify(user));
  },
  getUser: () => JSON.parse(localStorage.getItem("auth_user")),
  clear: () => {
    localStorage.removeItem("auth_token");
    localStorage.removeItem("auth_user");
  },
};
```


Explanation:

- This file is used for **API communication between frontend and backend**.
- Axios is used to create a **central API instance** with a base URL.
- The **API base URL** defines the backend server endpoint.
- An interceptor is used to:
 - Automatically attach JWT token
 - Send authenticated requests
- Token is stored in **localStorage** after login.
- The authStorage object manages:
 - Saving user session
 - Retrieving user data
 - Clearing session on logout
- This ensures:
 - Secure communication
 - Persistent login session
 - Clean and reusable API calls

SOFTWARE TESTING

Testing Implementation in the Project

Software testing was performed to ensure that the system works correctly, efficiently, and securely. In this project, testing was implemented at multiple levels to verify both backend APIs and frontend user interface.

The testing process was divided into two main approaches: automated testing and manual testing. Automated testing was used to validate backend API functionality, while manual testing was used to verify user interaction and interface behaviour.

For automated testing, a Python-based testing script was developed that sends HTTP requests to different API endpoints and checks the responses. The script validates status codes, authentication, role-based access, and data correctness.

For manual testing, the application was tested through a web browser by performing real user actions such as login, asking questions to the chatbot, submitting grievances, and accessing admin features.

This combined approach ensures that both the system logic and user experience are properly validated.

1. Implementation of Automated Testing

Automated testing was implemented in this project using a custom Python-based testing script. The script is designed to validate the backend APIs by sending HTTP requests and checking the responses automatically.

The testing script interacts with various API endpoints such as authentication, chat system, grievance management, feedback, notifications, and analytics. It verifies important aspects like response status codes, data structure, authentication tokens, and role-based access control.

The script is structured in a modular way where each function represents a specific test case. A common request handler is used to send API requests and parse responses. Assertions are used to validate expected outputs, ensuring that the system behaves correctly.

Working of the Testing Script

- The script sends HTTP requests using Python's request handling libraries
- It supports different request methods such as GET, POST, and PATCH
- It validates responses using:
 - Status code checking

- JSON structure validation
- Role-based access verification
- Authentication is tested using login APIs and JWT tokens
- Separate tests are defined for:
 - Normal user
 - Admin user
- Some test cases also perform database-level operations like:
 - Feedback submission
 - Grievance creation and update

Sample Test Code

```
• def test_backend_health() -> None:
•     response = request_json("GET", f"{HEALTH_URL}/")
•     assert_status(response, 200)
•     assert_has_keys(response.body, ["name", "status"])
•     assert response.body["status"] == "ok"
•
•
•
• def test_invalid_login_is_rejected() -> None:
•     response = request_json(
•         "POST",
•         api_url("/auth/login"),
•         {"email": "wrong-user@example.com", "password": "wrong-
password"},
•     )
•     assert_status(response, (400, 401, 422))
• def test_grievance_submission_and_lookup() -> None:
•     require_mutation_tests()
•     grievance_id = create_test_grievance()
•     response = request_json("GET",
api_url(f"/grievance/{grievance_id}"), token=get_user_token())
•     assert_status(response, 200)
•     assert_has_keys(response.body, ["id", "complaint", "status",
"comments"])
•     assert response.body["id"] == grievance_id
•
```

Only selected test code snippets are included for clarity, while the complete testing script covers all major modules and scenarios

Execution of Test Cases

The testing script is executed after starting the backend server. It automatically runs all defined test cases and prints the results.

Steps:

1. Start backend server
2. Run test script
3. View output in terminal

Test Execution Result

Automated Testing Output (All Test Cases Passed)

```
(venv) PS D:\FAQ_Chatbot>
>> $env:TEST_USER_EMAIL="aryaztech01@gmail.com"
>> $env:TEST_USER_PASSWORD="*****"
>> $env:TEST_ADMIN_EMAIL="admin@example.com"
>> $env:TEST_ADMIN_PASSWORD="****"
>> $env:RUN_MUTATION_TESTS="1"
>>
>> .\venv\Scripts\python.exe test.py
Running API tests against: http://localhost:8000/api
-----
PASS TC-01 Backend health check
PASS TC-02 OpenAPI schema available
PASS TC-03 Public chat validation
PASS TC-04 Protected chat sessions require login
PASS TC-05 Protected notifications require login
PASS TC-06 Invalid login is rejected
PASS TC-07 User login returns token
PASS TC-08 Admin login returns token
PASS TC-09 User chat sessions list
PASS TC-10 User notifications list
PASS TC-11 User grievances list
PASS TC-12 User blocked from analytics
PASS TC-13 User blocked from documents admin API
PASS TC-14 User blocked from expert queue
PASS TC-15 Logged-in chat validation
PASS TC-16 Feedback validation
PASS TC-17 Grievance validation
PASS TC-18 Admin analytics usage
PASS TC-19 Admin analytics failures
PASS TC-20 Admin documents list
PASS TC-21 Admin grievance queue
PASS TC-22 Admin expert queue
PASS TC-23 Feedback submission
PASS TC-24 Grievance submission and lookup
PASS TC-25 User grievance comment
PASS TC-26 Admin updates grievance status
-----
Summary: 26 passed, 0 skipped, 0 failed
(venv) PS D:\FAQ_Chatbot>
```

Result Analysis

- Total test cases executed: 26
- All test cases returned PASS status
- No failures or errors were observed
- System handled authentication, validation, and access control correctly

Conclusion of Automated Testing

The automated testing confirms that the backend APIs are functioning correctly and efficiently. The system successfully validates inputs, enforces security, and handles different user roles properly. This ensures that the backend is stable and reliable for real-world usage.

2. Implementation of Manual Testing

Manual testing was performed to evaluate the system from an end-user perspective. The objective of manual testing was to verify that all features of the application are working correctly and provide a smooth user experience.

In this project, manual testing was carried out by interacting with the system through a web browser. Different scenarios were tested by providing valid and invalid inputs to ensure that the system behaves as expected in all conditions.

The testing covered both functional and usability aspects of the application. Each module was tested individually and then tested together to verify proper integration.

Testing Environment

Manual testing was conducted in the following environment:

- Frontend running on local server (React application)
- Backend running on FastAPI server
- Database connected using PostgreSQL
- Browser used: Google Chrome

Modules Tested

The following modules were tested manually:

- User Registration and Login
- Public Chatbot Interface
- Authenticated Chat System
- Chat History and Sessions

- Document Upload (Admin)
- Grievance Submission and Tracking
- Expert Escalation System
- Notifications and Dashboard

Testing Procedure

The manual testing process followed these steps:

1. The backend server was started using FastAPI.
2. The frontend application was launched using React.
3. The application was accessed through a web browser.
4. Different functionalities were tested by performing user actions such as login, asking questions, uploading documents, and submitting grievances.
5. Both valid and invalid inputs were tested to check system validation.
6. Role-based testing was performed using normal user and admin accounts.

Test Scenarios

Test Case ID	Module	Test Scenario	Expected Result
TC-01	Authentication	User login with valid data	Login successful
TC-02	Authentication	Login with invalid password	Error message displayed
TC-03	Chatbot	Ask question without login	Answer is displayed
TC-04	Chatbot	Ask question after login	Accurate response generated
TC-05	Chat	Multiple queries	System handles all queries
TC-06	Chat History	Refresh page	Chat history is retained
TC-07	Document Upload	Upload PDF file	File uploaded successfully

TC-08	OCR	Upload image document	Text extracted correctly
TC-09	Grievance	Submit complaint	Complaint created
TC-10	Grievance	Track complaint	Status displayed
TC-11	Expert	Escalate query	Query sent to expert
TC-12	Dashboard	View analytics	Data displayed correctly
TC-13	Notifications	Open notifications	Notifications shown
TC-14	Logout	Logout user	Session cleared

Observations

- The chatbot responded correctly to most user queries.
- The system handled both authenticated and unauthenticated users properly.
- Grievance system successfully created and tracked complaints.
- Admin functionalities such as document upload and analytics worked as expected.
- No major errors or crashes were observed during testing.

Advantages of Manual Testing

- Helps in identifying real user experience issues
- Ensures UI and usability correctness
- Detects errors that automated testing may miss

Limitations of Manual Testing

- Time-consuming process
- Depends on human observation
- Not suitable for large-scale repetitive testing

Conclusion of Manual Testing

Manual testing confirms that the system is functioning correctly from the user's perspective. All modules are working as expected, and the application provides a smooth and user-friendly experience. The system is stable and ready for deployment.

CONCLUSION

The project titled “AI-Powered FAQ Chatbot for MPOnline Services” has been successfully designed and developed with the objective of providing a smart and efficient solution for handling user queries related to government services. In today’s digital environment, users often face difficulty in understanding procedures and finding correct information on government portals. This project addresses that gap by offering an intelligent chatbot that delivers instant and relevant responses.

The system makes use of modern web technologies along with artificial intelligence to create a seamless interaction experience. A key highlight of this project is the implementation of the Retrieval-Augmented Generation approach, where the chatbot does not rely solely on pre-trained knowledge but retrieves relevant information from a document-based knowledge base and then generates responses. This significantly improves the accuracy and reliability of the answers while reducing the chances of incorrect or misleading information.

During the development of this project, multiple modules were carefully designed and integrated to ensure complete functionality. These include user authentication and authorization, chatbot interaction system, document upload and OCR processing, chat history management, grievance handling system, expert escalation mechanism, feedback collection, notification system, and analytics dashboard. Each module plays a crucial role in enhancing the usability and effectiveness of the system.

The backend of the application was developed using FastAPI, which provides high performance and efficient API handling, while the frontend was built using React to ensure a dynamic and responsive user interface. PostgreSQL was used for managing structured data, and FAISS was used as a vector database to enable fast and accurate similarity search for the chatbot. The integration of these technologies ensures that the system is scalable, maintainable, and suitable for real-world applications.

Testing was an important part of this project. Both manual and automated testing approaches were used to validate system functionality. Manual testing ensured that the user interface and user experience are smooth and error-free, while automated testing verified the correctness of backend APIs and system logic. The results confirmed that the system is stable, responsive, and capable of handling different user scenarios effectively.

The project also focuses on security and reliability. User authentication is implemented using secure mechanisms such as password hashing and token-based authentication. Role-based

access control ensures that different types of users, such as normal users, administrators, and experts, have appropriate permissions within the system.

Overall, this project demonstrates the practical application of artificial intelligence in solving real-world problems. It simplifies access to government services, reduces dependency on manual assistance, and improves user satisfaction by providing quick and accurate information. The system is flexible and can be further enhanced with additional features, making it a strong foundation for future development.

In conclusion, the developed chatbot successfully meets all the defined objectives and provides an effective, scalable, and user-friendly solution for handling MPOnline-related queries. It has the potential to be deployed in real-world environments to improve digital service delivery and user experience.

BIBLIOGRAPHY

The following resources were referred to during the development of this project:

1. Official documentation of FastAPI – <https://fastapi.tiangolo.com/>
2. Official documentation of React – <https://react.dev/>
3. Official documentation of PostgreSQL – <https://www.postgresql.org/docs/>
4. Official documentation of FAISS – <https://faiss.ai/>
5. Official documentation of Tesseract OCR – <https://tesseract-ocr.github.io/>
6. API documentation of Google Gemini – <https://ai.google.dev/>
7. Online tutorials and references from Stack Overflow – <https://stackoverflow.com/>
8. General programming and conceptual references from GeeksforGeeks – <https://www.geeksforgeeks.org/>